

Coaching Tool for Public Engagement

Technical Architecture & System Prompt Reference
March 2026

This document covers the coaching tool's architecture, current deployment on Render.com, proposed AWS migration (Lambda-based), data pipeline (current & AWS), agent system, and all system prompts powering LLM interactions.

Contents

1. System Overview
2. Current Architecture
3. Tech Stack
4. Current Deployment (Render.com)
5. Proposed AWS Architecture
6. Data Pipeline (Current)
7. Data Pipeline on AWS (Admin Dashboard → Lambda)
8. Agent System & Tool Calling
9. Frontend Pages & Routes
10. API Endpoints
11. System Prompts – Detailed Reference
12. Environment Variables

1. System Overview

The Coaching Tool for Public Engagement is a **RAG (Retrieval-Augmented Generation)** application that helps practitioners plan, assess, and improve their community engagement projects. It combines a curated knowledge base of engagement guides, case studies, and best practices with an agentic LLM system that searches this knowledge base before every recommendation.

The tool offers two main user journeys: (1) a **Nesta Framework Self-Assessment** with AI coaching for each of 9 dimensions, and (2) a **Case Study Library** that scores real-world case studies for relevance to the user's situation. A general-purpose **Chatbot** is also available for ad-hoc questions about public engagement.

2. Current Architecture

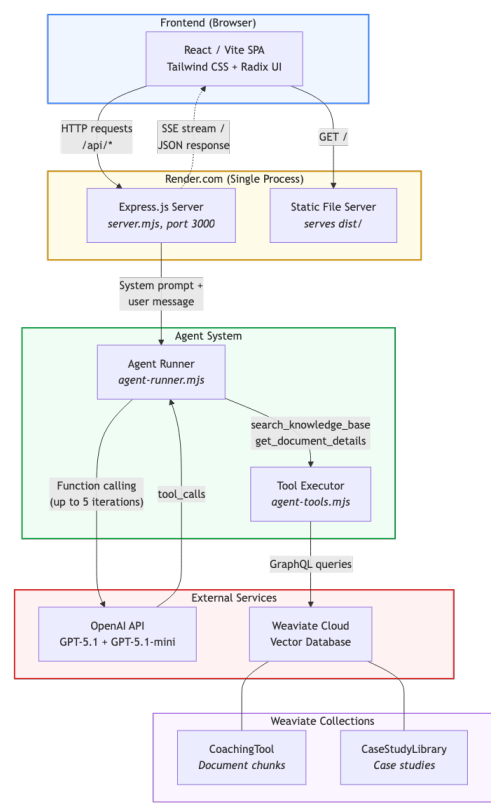


Figure 1: Current system architecture

3. Tech Stack

Layer	Technology	Details
Frontend	React 18 + Vite 6 + TypeScript	SPA with react-router v7, Tailwind CSS v4, Radix UI
Backend	Express.js 5 (Node 20+)	ES Modules, SSE streaming, agent loop
LLM	OpenAI GPT-5.1	Chat completions + function calling; mini for classification
Vector DB	Weaviate Cloud	text2vec-openai, BM25 + nearText hybrid search
Deployment	Render.com (current)	Auto-deploy from GitHub, free tier
Testing	Vitest + Testing Library	Unit tests for React components

4. Current Deployment (Render.com)

The app runs as a single web service on Render.com's free tier, configured via `render.yaml`. Vite builds the frontend to `dist/`, and Express serves both the static SPA and the API from one process.

Setting	Value
Service type	Web service (Node.js)

Setting	Value
Plan	Free tier
Auto-deploy	Yes (push to main)
Build	npm install && npm run build (vite build)
Start	node server.mjs
Node	20.12.2

Limitations: Free tier has cold starts, 512 MB RAM, no auto-scaling, and the server spins down after 15 min of inactivity.

5. Proposed AWS Architecture

The proposed migration moves the application to AWS for production-grade reliability, auto-scaling, and separation of concerns. The core API server runs on **ECS Fargate** (for long-running SSE connections), while document ingestion, classification, and background coaching evaluation are offloaded to **AWS Lambda** functions for zero-idle-cost, event-driven execution.

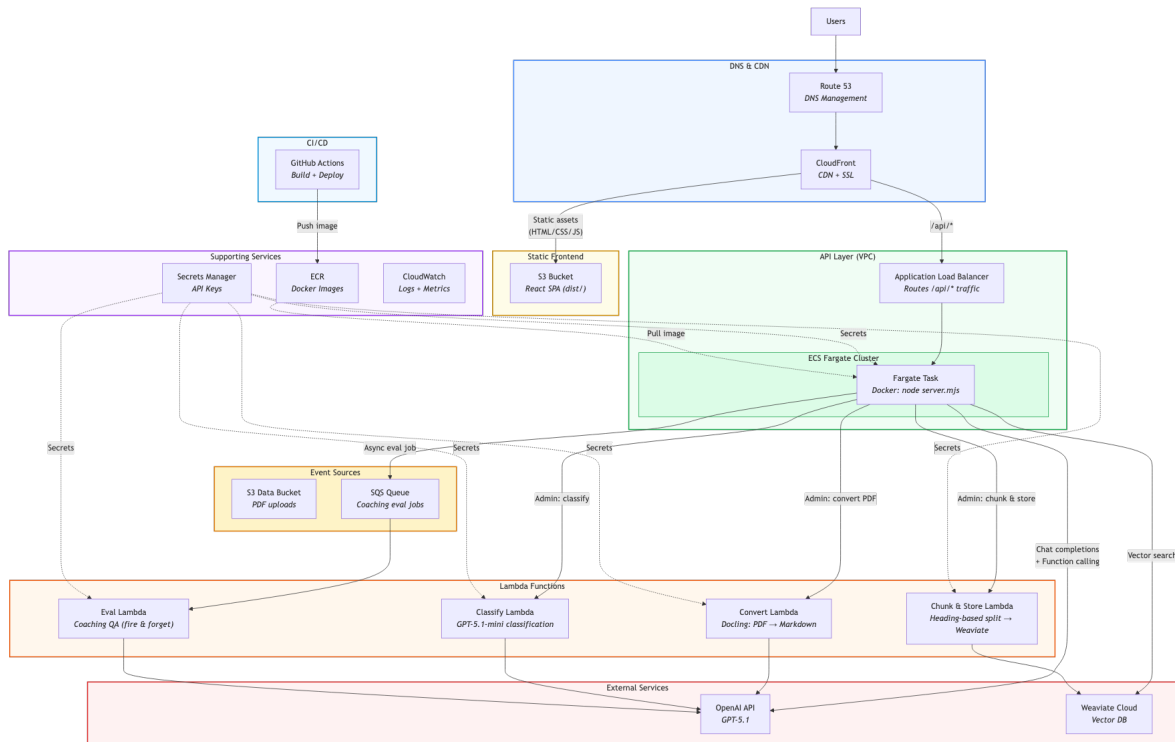


Figure 2: Proposed AWS architecture – ECS Fargate + Lambda

Component Breakdown

AWS Service	Role	Why
S3 + CloudFront	Static frontend hosting + CDN	Global edge caching, HTTPS, custom domain. Serves React SPA independently from the API.
ECS Fargate	API server (Express.js)	Serverless containers for the main API. Supports long-running SSE streaming connections that exceed Lambda's 15-min timeout.
ALB	Load balancer + routing	Routes /api/* to ECS, health checks, SSL termination. CloudFront origin for API traffic.
Lambda (Convert)	PDF → Markdown via Docling	Python Lambda with Docling layer. Converts uploaded PDFs to structured Markdown preserving headings, tables, and layout. Called by Express during admin upload flow.
Lambda (Classify)	Document classification	Invoked by the Express server when admin uploads a doc. Runs GPT-5.1-mini classification (content_type + summary). Keeps the main server lean.
Lambda (Chunk & Store)	Chunking + Weaviate storage	Heading-based chunking (lib/chunking.mjs logic). Splits Markdown on ##### boundaries, sub-splits oversized sections, adds 300-char overlap + contextual prefix. Stores in Weaviate.
Lambda (Eval)	Coaching QA evaluation	Fired asynchronously via SQS after each coaching message. Runs the evaluate-coaching prompt and logs scores. Offloads background work from the API server.

AWS Service	Role	Why
S3 (Data Bucket)	PDF storage	Stores uploaded PDF files. Express saves PDFs here before invoking the Convert Lambda.
SQS	Async job queue	Decouples coaching eval from the API server. Express sends a message, Lambda picks it up.
ECR	Docker image registry	Stores server Docker images. GitHub Actions pushes new images on merge to main.
Route 53	DNS management	Custom domain routing to CloudFront distribution.
Secrets Manager	API key storage	Securely stores OPENAI_API_KEY, WEAVIATE_API_KEY. Injected into ECS + Lambda at runtime.
CloudWatch	Monitoring + logs	Container logs, Lambda invocation metrics, error rate alarms, coaching-eval score tracking.

Why Lambda + ECS Fargate (Hybrid)

Concern	Lambda	ECS Fargate
Max duration	15 min	Unlimited
SSE streaming	Not supported (response timeout)	Native support
Cold start	~1–3s (Node.js)	None (always-on)
Pricing model	Pay per invocation + duration	Pay per vCPU-hour + memory-hour
Ideal for	Event-driven, short tasks: ingestion, classification, eval	Long-running API server with streaming responses

The chatbot's SSE streaming connections can last several minutes and Lambda's response timeout makes it unsuitable for the main API. ECS Fargate handles the always-on API server, while Lambda handles everything that's event-driven and short-lived.

Migration Path

- **Phase 1:** Containerize the server (Dockerfile), push to ECR, deploy on ECS Fargate behind ALB.
- **Phase 2:** Create 4 Lambda functions: Convert (Python + Docling), Classify, Chunk & Store, and Eval. Create S3 data bucket for PDF storage.
- **Phase 3:** Wire Express admin routes to invoke Lambdas via AWS SDK. Create SQS queue for coaching eval. Wire Express to send eval jobs to SQS → Eval Lambda.
- **Phase 4:** Move frontend to S3 + CloudFront. DNS cutover via Route 53. Secrets migration to Secrets Manager. CloudWatch dashboards.
- **Phase 5 (optional):** Migrate Weaviate to self-hosted on ECS or EC2 for cost control at scale.

Cost Estimate (Low Traffic)

Service	Estimated Monthly Cost
ECS Fargate (0.25 vCPU, 0.5 GB, always-on)	~\$9
ALB	~\$16 + data transfer
Lambda (all 4 functions)	< \$1 (Free tier: 1M requests + 400K GB-s)
SQS	< \$1 (Free tier: 1M requests)
S3 (data bucket + frontend)	< \$1
CloudFront	< \$1 (low traffic)
Secrets Manager	~\$1
CloudWatch	Free tier likely sufficient
ECR	< \$1
Total	~\$27–30/month

Note: Weaviate Cloud and OpenAI API costs are separate. Lambda functions stay in free tier at low traffic – the biggest cost savings vs. a second always-on Fargate task for ingestion.

6. Data Pipeline (Current)

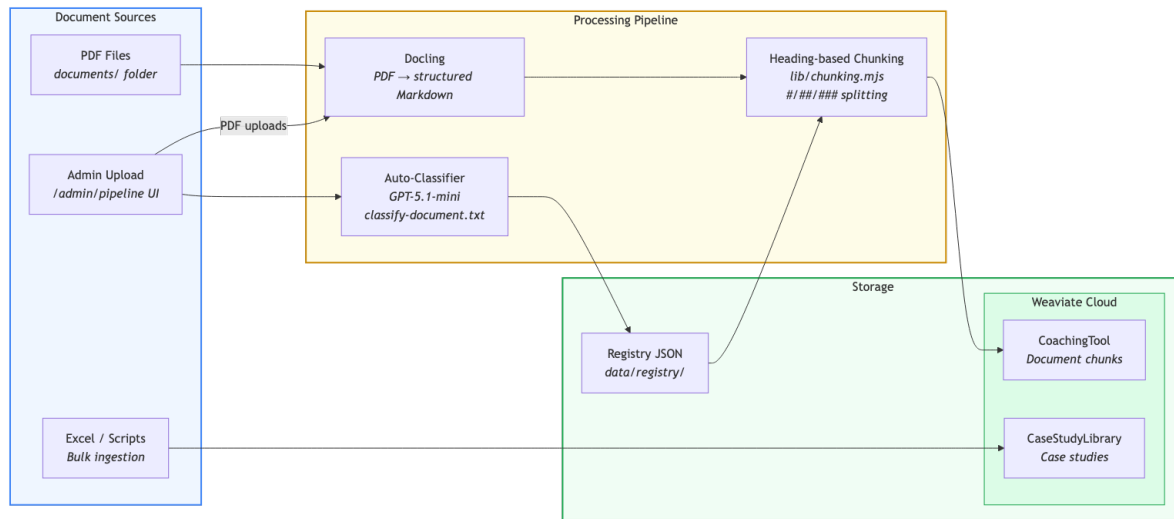


Figure 3: Current data pipeline

Path A: PDF Ingestion (CLI)

PDFs in documents/ are first converted to structured Markdown using **Docling**, then chunked via heading-based splitting. Stored in Weaviate CoachingTool collection.

- **PDF → Markdown** – scripts/convert-pdfs.py runs **Docling** (DocumentConverter) to convert PDFs to structured Markdown preserving headings, tables, and layout.
- **Heading-based chunking** – lib/chunking.mjs parses Markdown headings (###/###) and splits into semantic sections (max ~2000 chars).
- **Overlap & context** – ~300 char overlap between chunks + contextual retrieval prefix (document title + section path).
- **Embedding & storage** – Chunks embedded via text2vec-openai and stored in Weaviate.

Path B: Admin Dashboard Upload

Admin UI at /admin/pipeline for uploading documents. Auto-classified using the classify-document.txt prompt (GPT-5.1-mini). Registry JSON stored in data/registry/.

Weaviate Collections

Collection	Purpose	Key Fields
CoachingTool	Document chunks	content, document_id, doc_name, source_url, content_type, section_name, chunk_index
CaseStudyLibrary	Case studies	document_id, title, summary, location, timeframe, demographic, scale, tags, key_outcomes

7. Data Pipeline on AWS (Admin Dashboard → Lambda)

On AWS, the admin dashboard's document ingestion is offloaded to **Lambda functions**. Each step of the pipeline – PDF conversion (Docling), classification (GPT-5.1-mini), and chunking + storage (Weaviate) – runs in a dedicated Lambda, keeping the Express API server lean and the pipeline fully serverless.

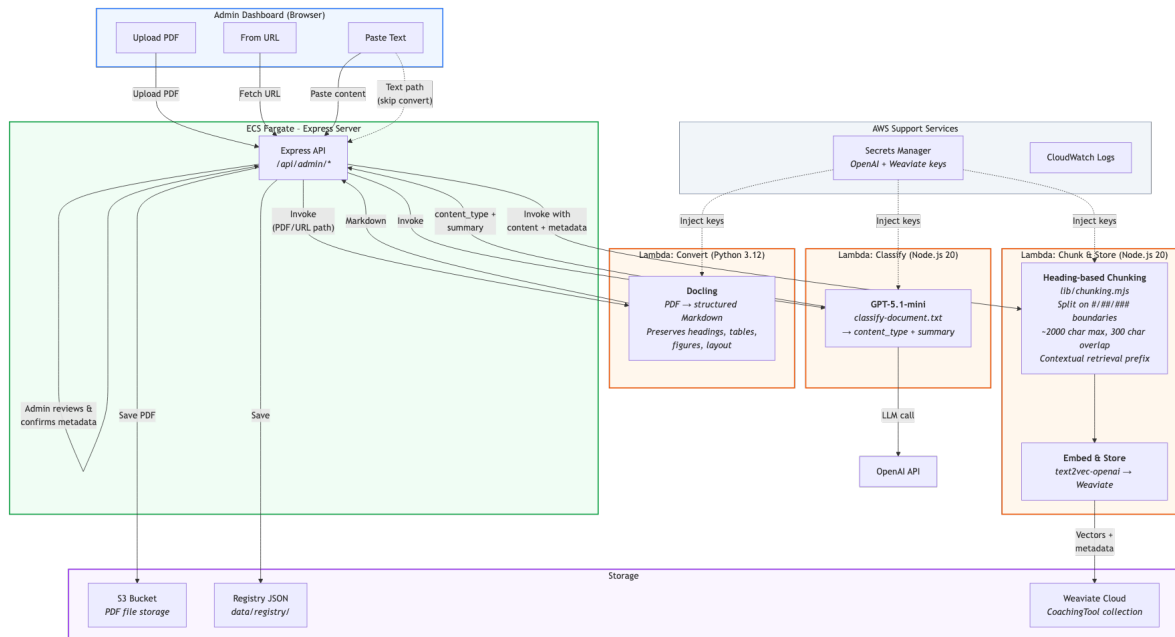


Figure 4: Admin dashboard data pipeline on AWS

Admin Upload Flow (Step by Step)

The admin dashboard at `/admin/pipeline` supports three ingestion modes: **Upload PDF**, **From URL**, and **Paste Text**. On AWS, each backend operation is handled by a Lambda function invoked from the Express server via the AWS SDK.

Step 1: PDF Conversion – Convert Lambda (Python 3.12)

- Admin uploads a PDF (or provides a PDF URL) via the dashboard.
- Express saves the PDF to the **S3 data bucket** and invokes the **Convert Lambda**.
- The Lambda runs **Docling** (`docling.document_converter.DocumentConverter`) to convert PDF to **structured Markdown**.
- Docling preserves document structure: headings, tables, figures, lists, and page layout.
- The resulting Markdown is returned to Express, which sends it to the admin for review.
- *For text paste / HTML URL paths, this step is skipped – content goes directly to Step 2.*
- **Runtime:** Python 3.12 (Docling + dependencies as Lambda layer or container image) | **Memory:** 1024 MB | **Timeout:** 5 min.

Step 2: Classification – Classify Lambda (Node.js 20)

- Admin reviews the converted content, edits metadata (title, source, date).
- Express invokes the **Classify Lambda** with the document name, source, and first 2000 chars.
- The Lambda calls **GPT-5.1-mini** with the `classify-document.txt` prompt (temp 0).
- Returns `{content_type, summary}` – one of: `case_study`, `transcript`, `blog_post`, `journal_article`, `report`, `guide`, `policy_brief`, `lecture`, `tool_or_resource`, other.
- Express saves a **registry JSON** entry to `data/registry/`.
- **Runtime:** Node.js 20 | **Memory:** 256 MB | **Timeout:** 30s.

Step 3: Chunk & Store – Chunk Lambda (Node.js 20)

- Express invokes the **Chunk & Store Lambda** with the full content + metadata.
- The Lambda runs **heading-based chunking** (same logic as `lib/chunking.mjs`):
 - Parses Markdown headings (`#`, `##`, `###`, `####`) to identify semantic sections.
 - Splits on heading boundaries into coherent chunks (max ~2000 chars).
 - Sub-splits oversized sections at paragraph boundaries.
 - Adds ~300 char overlap between consecutive chunks.
 - Prepends a **contextual retrieval prefix** (document title + section path) to each chunk.
- Chunks are sent to **Weaviate Cloud**, where `text2vec-openai` generates embeddings.
- Stored in `coachingTool` collection with full metadata (`document_id`, `doc_name`, `content_type`, `section_name`, `chunk_index`).
- **Runtime:** Node.js 20 | **Memory:** 512 MB | **Timeout:** 5 min.

Coaching Eval (Background – Express → SQS → Eval Lambda)

- After each coaching message, Express sends a job to an **SQS queue** with the transcript + context.
- **Eval Lambda** polls the queue, runs `evaluate-coaching.txt` (GPT-5.1, temp 0). Scores logged to CloudWatch.
- Fully async – the user never waits for this. Decouples QA from the chat response.
- **Runtime:** Node.js 20 | **Memory:** 256 MB | **Timeout:** 1 min.

Lambda Function Summary

Lambda	Runtime	Memory	Timeout	Trigger	What It Does
Convert	Python 3.12	1024 MB	5 min	Express invoke	Docling: PDF → structured Markdown
Classify	Node.js 20	256 MB	30s	Express invoke	GPT-5.1-mini: content_type + summary
Chunk & Store	Node.js 20	512 MB	5 min	Express invoke	Heading-based split → Weaviate
Eval	Node.js 20	256 MB	1 min	SQS queue	Coaching QA scores → CloudWatch

Key Differences from Current Setup

Aspect	Current (Render / Local)	AWS (Lambda)
PDF conversion	Docling runs in Express process (blocks server)	Convert Lambda – isolated Python 3.12 with Docling
Classification	GPT-5.1-mini call in Express process	Classify Lambda – isolated, auto-scales
Chunking	Node.js in Express or CLI script	Chunk & Store Lambda – same logic, isolated
Coaching eval	Fire-and-forget in Express process	SQS → Eval Lambda – fully decoupled
Secrets	Env vars in Render dashboard	Secrets Manager – injected at runtime, rotatable
Monitoring	Console logs only	CloudWatch Logs – per-Lambda, searchable, alertable
Cost when idle	Free (runs locally)	\$0 – Lambda only charges per invocation
Impact on API	Heavy ops block the API server	Fully isolated – zero API impact

8. Agent System & Tool Calling

Most prompts are **agentic** – they use OpenAI function calling to search Weaviate before generating responses, ensuring recommendations are grounded in actual documents.

- **runAgentLoop()** – Full loop, returns final text. Used by non-streaming endpoints.
- **resolveAgentToolCalls()** – Resolves tools, returns messages for streaming. Used by SSE endpoints.
- Up to **5 iterations**. On the last, `tool_choice: "none"` forces text output.

Tool	Description	Parameters
search_knowledge_base	Semantic search over CoachingTool. BM25 first, nearText fallback.	query (required), content_type_filter (optional)
get_document_details	Retrieve all chunks for a document by ID.	document_id (required)

9. Frontend Pages & Routes

Route	Component	Description
/	Home	Landing page with nav cards to Coach and Case Studies
/coach	Coach	Nesta self-assessment: 9 free-text questions + scenario generator
/coach/dashboard	AssessmentDashboard	Status cards (addressed/partial/not-addressed) + coaching chat
/coach/reflection	Reflection	Final reflection after assessment + coaching

Route	Component	Description
/case-studies	CaseStudies	Browse/search case study library with relevance scoring
/case-studies/:id	CaseStudyDetail	Individual case study detail view
/admin	AdminDashboard	Knowledge base management dashboard
/admin/documents	DocumentExplorer	Browse documents in Weaviate
/admin/pipeline	PipelineManager	Upload, classify, and ingest documents

A general-purpose **ChatBot** component is accessible from every page. **CoachingChatPanel** is embedded in the AssessmentDashboard.

10. API Endpoints

Method	Endpoint	Prompt	Response
POST	/api/chatbot	chatbot.txt (+ coaching-context.txt)	SSE stream (markdown + sources)
POST	/api/evaluate-assessment	evaluate-assessment.txt	JSON {evaluations[]}
POST	/api/analyze-cross-resolution	cross-resolution.txt	JSON {currentQuestionResolved, resolvedQuestionIds[]}
POST	/api/generate-reflection	generate-reflection.txt	JSON {reflection{}}
POST	/api/score-case-studies	score-case-studies.txt	JSON {scores[]}
POST	/api/generate-scenario-responses	generate-scenario.txt	JSON {scenario, responses{}}
GET	/api/case-studies	(direct Weaviate query)	JSON array of case studies
POST	/api/admin/classify	classify-document.txt	JSON {content_type, summary}

The evaluate-coaching.txt prompt runs internally after each coaching message (fire-and-forget on Render; SQS → Lambda on AWS).

11. System Prompts – Detailed Reference

All system prompts are stored as plain text files in the prompts/ folder and loaded at server startup via prompts/load.mjs. Prompts can be edited without touching any code – just modify the .txt file and restart the server.

File	Constant Name	Prompt Type
chatbot.txt	CHATBOT_PROMPT	Agentic (tools + SSE streaming)
score-case-studies.txt	SCORE_CASE_STUDIES_PROMPT	Agentic (tools, JSON output)
evaluate-assessment.txt	EVALUATE_ASSESSMENT_PROMPT	Agentic (tools, JSON output)
coaching-context.txt	COACHING_CONTEXT_PREFIX	Template (client-side, injected into chatbot)
cross-resolution.txt	CROSS_RESOLUTION_PROMPT	Direct LLM call (no tools, JSON output)
generate-reflection.txt	GENERATE_REFLECTION_PROMPT	Agentic (tools, JSON output)
evaluate-coaching.txt	EVALUATE_COACHING_PROMPT	Direct LLM call (background, JSON output)
generate-scenario.txt	GENERATE_SCENARIO_PROMPT	Agentic (tools, JSON output)
scenario-descriptions.json	SCENARIO_DESCRIPTIONS	Data file (6 persona definitions)
classify-document.txt	CLASSIFY_SYSTEM	Direct LLM call (GPT-5.1-mini, JSON output)

11.1 Chatbot (General Assistant)

File: prompts/chatbot.txt | **Endpoint:** POST /api/chatbot | **Type:** Agentic + SSE

Purpose: General-purpose chatbot. Users ask any question about public engagement and get evidence-grounded answers. Also handles coaching chats when COACHING_CONTEXT_PREFIX is detected.

Input: User message + conversation history. **Output:** Streamed markdown with "### Sources" section.

Key Behaviors

- Must search KB at least once before answering
- Every recommendation cites source inline: [Source: Document Name]

- Admits gaps: “Based on available resources, I don’t have specific guidance on this.”

11.2 Score Case Studies (Relevance)

File: prompts/score-case-studies.txt | **Endpoint:** POST /api/score-case-studies | **Type:** Agentic + JSON

Purpose: Scores every case study in the library against the user’s situation using a weighted 6-dimension rubric (100 points total).

Input: Practitioner context (10 fields) + list of case studies. **Output:** JSON scores array with dimension breakdown.

Scoring Rubric

Dimension	Max Points
Issue/Topic Alignment	20
Engagement Goal Match	20
Audience/Demographic Fit	15
Timeline & Scale Compatibility	15
Resource & Constraint Alignment	15
Stuck Point / Process Stage Relevance	15

Must search KB at least 4 times. Always returns at least 5 case studies sorted by score.

11.3 Evaluate Assessment (Nesta Framework)

File: prompts/evaluate-assessment.txt | **Endpoint:** POST /api/evaluate-assessment | **Type:** Agentic + JSON

Purpose: Evaluates practitioner’s responses to 9 Nesta questions as “addressed,” “partial,” or “not-addressed.” Gateway to coaching flow.

Input: 9 Nesta questions + user responses. **Output:** JSON with 9 evaluations (questionId, status, gap, coachingContext).

The 9 Nesta Questions

- Q1: Have you articulated the project’s goals?
- Q2: Have you identified the right participants?
- Q3: Can you reach the participants you identified?
- Q4: Who is the right owner?
- Q5: Have you included incentives for participation?
- Q6: Have you defined the tasks?
- Q7: Have you established the workflow?
- Q8: How will you evaluate inputs?
- Q9: How will you use what the group creates?

Decision Test

- “addressed” – Names specific actions, people, methods, or timelines. Actionable without follow-up.
- “not-addressed” – Fails 2+ of: specific method? specific people? concrete step? on-topic?
- “partial” – Fails exactly 1 check. Does NOT default to partial for vague responses.

11.4 Coaching Context (Socratic Coach)

File: prompts/coaching-context.txt | **Used by:** CoachingChatPanel.tsx (client-side) | **Type:** Template

Purpose: Turns the general chatbot into a Socratic coach for a specific Nesta question. Injected as first message in coaching conversations.

Placeholders: {{question}}, {{userResponse}}, {{status}}, {{gap}}

Coaching Approach

- Starts by acknowledging what the user did well
- Socratic questioning – probing questions, not lectures
- Evaluates user proposals against the identified gap
- If proposal is strong, explicitly tells them to mark as resolved

11.5 Cross-Resolution Analyzer

File: prompts/cross-resolution.txt | **Endpoint:** POST /api/analyze-cross-resolution | **Type:** Direct LLM (no tools)

Purpose: After coaching about one Nesta question, determines if the conversation also resolved gaps from other unresolved questions. Prevents repetition across sessions.

Input: Transcript + current question gap + other unresolved cards. **Output:** JSON {currentQuestionResolved, resolvedQuestionIds[]}

Key Behaviors

- Deliberately conservative – requires concrete discussion that closes the gap
- Passing mentions and vague overlap do NOT count
- Only returns IDs of OTHER cards (not the current question)

11.6 Generate Reflection (Final Summary)

File: prompts/generate-reflection.txt | **Endpoint:** POST /api/generate-reflection | **Type:** Agentic + JSON

Purpose: Generates comprehensive final reflection after assessment + coaching, summarizing the journey, strengths, gaps, and priority next steps.

Input: All 9 responses + evaluations + coaching chat histories (classified as PRODUCTIVE / RESOLVED WITHOUT CONVERSATION / UNRESOLVED / NO SESSION).

Output: JSON reflection with: summary (2–3 paragraphs), addressed[], partial[] (with nextSteps), notAddressed[] (with nextSteps), priorityActions[3] (action, rationale, timeline).

Key Behaviors

- Accounts for full journey – original answers AND coaching conversations
- Acknowledges growth in productive conversations
- Flags when users skipped coaching despite gaps
- Exactly 3 priority actions with suggested timeframes

11.7 Evaluate Coaching (Background QA)

File: prompts/evaluate-coaching.txt | **Endpoint:** None (fire-and-forget; SQS → Lambda on AWS) | **Type:** Direct LLM, temp 0

Purpose: Internal QA. Runs after every coaching message, logs results to server console (or CloudWatch on AWS). User never sees this.

Input: Nesta question, original response, gap, full transcript. **Output:** JSON with 6 dimension scores (1–10), overall 0–100, strengths, weaknesses.

Dimensions: socratic_approach, specificity, evidence_use, warmth_and_tone, gap_targeting, progression.

11.8 Generate Scenario (Test Data)

File: prompts/generate-scenario.txt + scenario-descriptions.json | **Endpoint:** POST /api/generate-scenario-responses

Purpose: Testing tool. Generates realistic responses to 9 Nesta questions based on a persona scenario.

Input: Scenario key or custom description. **Output:** JSON with scenario label + 9 responses.

Built-in Personas

Scenario	Description
well-prepared	Thorough planner with clear goals and workflows
vague-minimal	Generic, minimal-effort responses
contradictory	Subtle internal contradictions between answers
off-topic	Confuses engagement with marketing; answers wrong questions
over-ambitious	Grand plans with minimal resources
hostile-resistant	Skeptical; forced by management

11.9 Classify Document (Admin)

File: prompts/classify-document.txt | **Endpoint:** POST /api/admin/classify | **Type:** Direct LLM (GPT-5.1-mini, temp 0)

Purpose: Auto-classifies uploaded documents into a content type and generates a summary. Uses cheaper model. On AWS, runs in a dedicated **Classify Lambda**.

Input: Document name, source label, first 2000 chars. **Output:** JSON {content_type, summary}.

Types: case_study, transcript, blog_post, journal_article, report, guide, policy_brief, lecture, tool_or_resource, other.

12. Environment Variables

Variable	Req	Description	Default
WEAVIATE_HOST	Yes	Weaviate cluster hostname or URL	localhost:8080
WEAVIATE_API_KEY	Yes*	Weaviate API key (* not needed for local Docker)	(none)
WEAVIATE_SCHEME	No	http or https	https
OPENAI_API_KEY	Yes	OpenAI API key for GPT and embeddings	(none)
CHATBOT_MODEL	No	Model name for all agentic prompts	gpt-5.1
CHATBOT_MAX_RESULTS	No	Max search results per KB query	5
PORT	No	Server port	3000

Loaded from .env file via dotenv. Template at .env.example. On Render, configured in render.yaml. On AWS: ECS task uses Secrets Manager; Lambda functions use Secrets Manager or SSM Parameter Store.